

系统开发工具基础

肖扬

计算机科学与技术

学号：25020007140

2026 年 9 月 14 日

目录

1	实验环境说明	1
2	课堂学习部分	2
2.1	第 13 题建立可执行的本地质量门禁	2
2.2	第 14 题让 Make 只重建真正受影响的产物	6
2.3	第 15 题把本地 API 数据转换为可读报告	8
2.4	第 16 题修复并交付一个陌生的小型工具仓库	12
3	课后习题	18
3.1	第 1 题使用 Shell 创建并查看文件	18
3.2	第 2 题使用管道统计文本内容	19
3.3	第 3 题查看环境变量和命令返回码	21
3.4	第 4 题使用 Vim 编辑文本文件	23
3.5	第 5 题根据报错修复 Python 程序	24
3.6	第 6 题使用 time 查看 Python 程序运行时间	26
3.7	第 7 题创建并安装简单的 Python 包	28
3.8	第 8 题使用编程智能体生成 README	29
3.9	第 9 题编写简单的 Bug 报告	33
3.10	第 10 题为加法函数编写单元测试	35
4	解题感悟	38
4.1	课上练习	38
4.2	课后习题	38
5	上传 GitHub	40

5.1	编写并更新.gitignore	40
5.2	GitHub 仓库链接	41
5.3	commit 提交记录	41

Chapter 1

实验环境说明

1. 系统环境：Ubuntu，用户名 xy_ai，WSL2 架构
2. Shell：bash 5.1.16 (x86_64-pc-linux-gnu)
3. Python：Miniconda base 环境中的 Python 3.12.4
4. 代码检查工具：Ruff 0.16.6
5. 测试工具：Pytest 9.1.1
6. LaTeX：TeX Live 2022/dev (Debian 版)，使用 XeLaTeX 编译

Chapter 2

课堂学习部分

2.1 第 13 题建立可执行的本地质量门禁

第 13 题 建立可执行的本地质量门禁

主题：代码质量 建议用时：12—15 分钟

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

题目要求

- 复制 q10 为 q13；在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。
- 编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest；运行并保证返回

图 2.1: 第 13 题题目要求

1. 复制项目并配置工具

在 assignment-4 目录中把第 10 题的项目复制为 q13，进入项目后检查原有的 pyproject.toml，再补充 Ruff 和 Pytest 的最小配置。

```
cp -r ../assignment-3/q10 q13
cd q13
nano pyproject.toml
```

新增的配置如下：

```
[tool.ruff]
```

```
line-length = 88
```

```
[tool.pytest.ini_options]
```

```
testpaths = ["tests"]
```

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cp -r ../assignment-3/q10 q13
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd q13
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ls
ai_log.md  dist  pyproject.toml  src  tests
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ cat pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007140"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ nano pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ nano pyproject.toml
```

图 2.2: 复制 q10 并修改 pyproject.toml

2. 补充两个测试

在 tests/test_cli.py 中保留空白姓名测试，并增加正常姓名测试。正常姓名测试运行 main() 后读取标准输出，检查输出内容中包含姓名 Xiao。

```
def test_normal_name(monkeypatch, capsys):
    monkeypatch.setattr(
        "sys.argv",
        ["sdt-greet", "--name", "Xiao"],
    )

    main()

    captured = capsys.readouterr()
    assert "Xiao" in captured.out
```

运行 pytest 后共收集两个测试，结果为 2 passed。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ cat tests/test_cli.py
import sys
import pytest

from greetlab.cli import main

def test_blank_name_exits(monkeypatch):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
    with pytest.raises(SystemExit) as exc:
        main()

    assert exc.value.code == 2
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ nano tests/test_cli.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ pytest
===== test session starts =====
platform linux -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/xy_ai/系统工具开发基础/assignment-4/q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
```

图 2.3: 补充正常姓名测试并运行 Pytest

3. 运行 Ruff 并修复问题

先运行 `ruff format` . 格式化项目, 再运行 `ruff check` . 检查代码。第一次检查提示 `tests/test_cli.py` 的导入顺序不正确。按照提示调整导入之间的空行后, 再次运行检查, 终端显示 `All checks passed!`。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ruff format
2 files reformatted, 2 files left unchanged
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ruff check
I001 [*] Import block is un-sorted or un-formatted
--> tests/test_cli.py:1:1

1 | / import sys
2 | | import pytest
3 | |
4 | | from greetlab.cli import main
  | |-----^
help: Organize imports
1 | import sys
2 | +
3 | import pytest
  |

Found 1 error.
[*] 1 fixable with the `--fix` option.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ nano tests/test_cli.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ruff check
All checks passed!
```

图 2.4: 运行 Ruff 并修复导入顺序问题

4. 编写并运行质量检查脚本

创建 `check.sh`, 按题目要求依次执行格式检查、代码检查和测试:

```
#!/usr/bin/env bash
```

```
set -e

ruff format --check .
ruff check .
pytest
```

第一次执行 `./check.sh` 时提示 `Permission denied`。使用 `chmod +x check.sh` 增加执行权限后重新运行，三个检查依次通过。最后执行 `echo $?`，返回值为 0。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ nano check.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ruff format --check
.
4 files already formatted
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ruff check .
All checks passed!
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ pytest
===== test session starts =====
platform linux -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/xy_ai/系统工具开发基础/assignment-4/q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ./check.sh
-bash: ./check.sh: Permission denied
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ chmod +x check.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/xy_ai/系统工具开发基础/assignment-4/q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ echo $?
0
```

图 2.5: 运行 `check.sh` 并确认返回值为 0

2.2 第 14 题让 Make 只重建真正受影响的产物

第 14 题 让 Make 只重建真正受影响的产物

主题：元编程：构建系统 建议用时：12—15 分钟

在 q14 中按给定内容创建三个源文件，只需完成 Makefile。

给定内容

```
# data.csv
name,value
alpha,2
beta,3
gamma,5

# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))

# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")

# report.md
# Data Report
```

题目要求

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。

第 10 页

系统开发工具基础·每周课上实验检查题

- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

图 2.6: 第 14 题题目要求

1. 创建题目给出的源文件

在 assignment-4 下建立 q14 目录，并按题目内容创建 data.csv、stats.py、build_report.py 和 report.md。

```
mkdir q14
cd q14
nano data.csv
nano stats.py
nano build_report.py
```

```
nano report.md
```

统计脚本读取 CSV 文件中的 value 列，把总和写入中间文件。报告脚本读取报告标题和统计结果，再生成最终报告。

2. 编写 Makefile

创建 Makefile，写明默认目标、中间产物、最终产物和清理目标之间的依赖关系：

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: report.md stats.txt build_report.py
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
```

all 和 clean 被声明为 PHONY 目标。生成 stats.txt 需要 data.csv 和 stats.py；生成 report.txt 需要 report.md、stats.txt 和 build_report.py。

3. 验证首次构建和无改动构建

第一次运行 make 时，终端依次执行：

```
python3 stats.py
python3 build_report.py
```

执行后生成 stats.txt 与 report.txt。其中统计结果为 10，最终报告包含标题和 Total: 10。没有修改文件时再次运行 make，终端显示：

```
make: Nothing to be done for 'all'.
```

4. 验证依赖更新和清理目标

使用 touch data.csv 更新数据文件的时间，再运行 make，两个生成步骤都会重新执行。随后直接运行两个 Python 脚本，再执行 make 时仍按文件依赖和修改时间判断。最后运行 make clean，只删除 stats.txt 和 report.txt，源文件与 Makefile 均被保留。

```

(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q13$ cd ~/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir q14
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd q14
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ nano data.csv
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ nano stats.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ nano build_report.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ nano report.md
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ nano Makefile
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ make
python3 stats.py
python3 build_report.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ ls
Makefile      data.csv      report.txt    stats.txt
build_report.py report.md      stats.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ ^C
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ cat stats.txt
10
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ cat report.txt
# Data Report

Total: 10
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ make
make: Nothing to be done for 'all'.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ touch data.csv
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ make
python3 stats.py
python3 build_report.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ python3 stats.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ python3 build_report.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ make clean
rm -f stats.txt report.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ ls
Makefile build_report.py data.csv report.md stats.py

```

图 2.7: 验证 Make 的增量构建和 clean 目标

2.3 第 15 题把本地 API 数据转换为可读报告

第 15 题 把本地 API 数据转换为可读报告

主题: 大杂烩: API、jq、CLI 约定与 Markdown 建议用时: 12—15 分钟

在 q15 中创建给定 JSON 和本地 HTTP 服务, 再用 curl 与 jq 生成简短 Markdown 报告。

给定内容

```

[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]

```

题目要求

- 把给定 JSON 保存为 packages.json, 并在 q15 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录, 并按 downloads 降序、name 升序排列。
- 编写 `api_report.sh`, 把结果生成 `summary.md`, 包含标题和 name/version/downloads 三列表格。

图 2.8: 第 15 题题目要求

1. 创建 JSON 文件并启动本地服务

在 assignment-4 下建立 q15 目录，把题目给出的 JSON 数据保存为 packages.json。随后在该目录启动端口为 8000 的本地 HTTP 服务：

```
mkdir q15
cd q15
nano packages.json
python -m http.server 8000
```

终端记录显示浏览器或 curl 请求 /packages.json 时返回状态码 200。完成请求后使用 Ctrl-C 停止服务。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q14$ cd ~/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir q15
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd q15
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ nano packages.json
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ python -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [14/Sep/2026 16:37:34] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 16:37:46] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 16:37:50] "GET /packages.json HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ cd ..
rm -r q15^C
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ cd ..
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ rm -r q15
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ ls
q13 q14 实验报告
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir q15
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd q15
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ nano packages.json
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ ls
packages.json
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ python -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [14/Sep/2026 16:42:30] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 16:42:47] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 16:44:42] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 16:45:12] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 16:45:49] "GET /packages.json HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ |
```

图 2.9: 创建 packages.json 并启动本地 HTTP 服务

2. 使用 curl 获取数据

保持本地 HTTP 服务运行，在另一个终端进入 q15，执行：

```
curl --noproxy 127.0.0.1 -fsS \
http://127.0.0.1:8000/packages.json
```

终端成功输出 packages.json 中的五条记录。

```
xy_ai@Lenovo: ~/系统工具开发 x xy_ai@Lenovo: ~/系统工具开发 x + v - □ x
(base) xy_ai@Lenovo:~$ cd ~/系统工具开发基础/assignment-4/q15
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ curl --noproxy 127.0.0.1 -fsS http://127.0.0.1:8000/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 400, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ curl --noproxy 127.0.0.1 -fsS http://127.0.0.1:8000/packages.json | jq '[.] | select(.status == "active" and .downloads >= 100)] | sort_by(-.downloads, .name)'
Command 'jq' not found, but can be installed with:
sudo apt install jq
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ sudo apt update
[sudo] password for xy_ai:
Get:1 file:/var/cuda-repo-wsl-ubuntu-13-3-local InRelease [1572 B]
Get:1 file:/var/cuda-repo-wsl-ubuntu-13-3-local InRelease [1572 B]
```

图 2.10: 使用 curl 获取本地 API 数据

3. 安装 jq

第一次把 curl 输出传给 jq 时，终端提示 jq: command not found。执行下面的命令更新软件包列表并安装 jq：

```
sudo apt update
sudo apt install jq
jq --version
```

安装完成后运行版本检查，结果为 jq-1.6。

```
xy_ai@Lenovo: ~/系统工具开发 x xy_ai@Lenovo: ~/系统工具开发 x + v - □ x
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ sudo apt install jq
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libjq1 libonig5
The following NEW packages will be installed:
  jq libjq1 libonig5
0 upgraded, 3 newly installed, 0 to remove and 230 not upgraded.
Need to get 359 kB of archives.
After this operation, 1087 kB of additional disk space will be used.
```

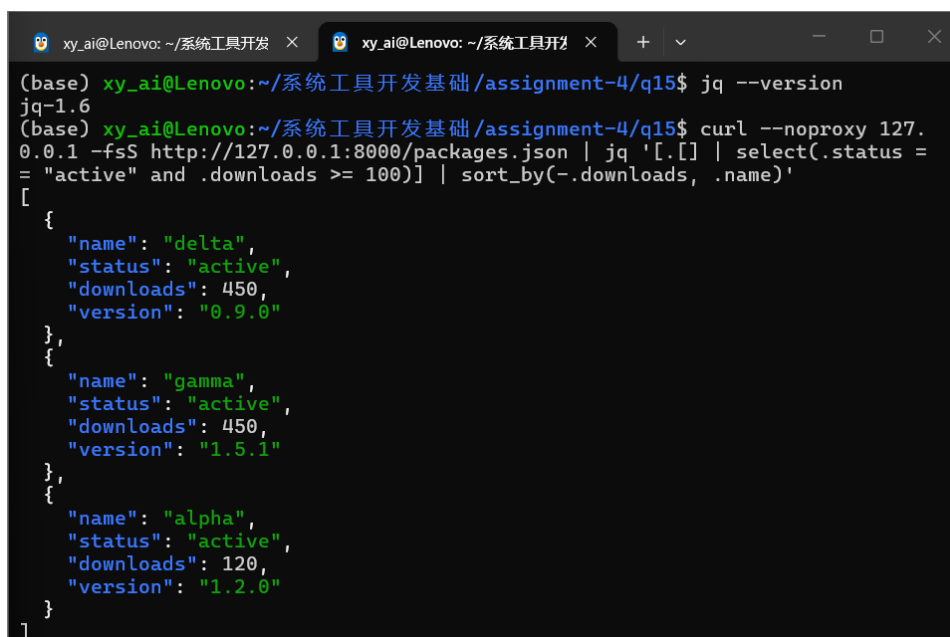
图 2.11: 安装 jq 工具

4. 筛选并排序记录

使用 jq 保留 status 为 active 且 downloads 不少于 100 的记录，再按 downloads 降序、name 升序排列：

```
curl --noproxy 127.0.0.1 -fsS \
  http://127.0.0.1:8000/packages.json |
jq '[.] | select(.status == "active" and .downloads >= 100)]
  | sort_by(-.downloads, .name)'
```

输出顺序为 delta、gamma、alpha。delta 和 gamma 的 downloads 都是 450，因此按 name 升序排列；alpha 的 downloads 为 120，排在最后。



```
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q15$ jq --version
jq-1.6
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q15$ curl --noproxy 127.0.0.1 -fsS http://127.0.0.1:8000/packages.json | jq '[.[] | select(.status == "active" and .downloads >= 100)] | sort_by(-.downloads, .name)'
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
```

图 2.12: 使用 jq 筛选并排序 JSON 记录

5. 编写报告生成脚本

创建 api_report.sh，在 jq 中把筛选结果转换为 Markdown 表格，并将结果重定向到 summary.md：

```
#!/usr/bin/env bash

set -e

curl --noproxy 127.0.0.1 -fsS \
  http://127.0.0.1:8000/packages.json |
jq -r '
  [.[] | select(.status == "active" and .downloads >= 100)]
  | sort_by(-.downloads, .name)
  | "# Package Summary\n\n| name | version | downloads |\n
|---|---|---:|\n"
  + (map("| \(.name) | \(.version) | \(.downloads) |")
    | join("\n"))
' > summary.md
```

使用 `chmod +x api_report.sh` 增加执行权限后运行脚本。生成的报告包含标题和 name、version、downloads 三列表格。再次运行脚本后内容保持不变。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ nano api_report.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ chmod +x api_report.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ ./api_report.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ cat summary.md
# Package Summary

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ ls
api_report.sh  packages.json  summary.md
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ ./api_report.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q15$ cat summary.md
# Package Summary

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 2.13: 运行脚本并检查重复生成的 Markdown 报告

2.4 第 16 题修复并交付一个陌生的小型工具仓库

第 16 题 修复并交付一个陌生的小型工具仓库

主题: 课堂综合测试 建议用时: 12—15 分钟

复用第 13 题的项目, 在 q16 中完成一次 15 分钟综合检查。

题目要求

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 Hello, name!, 运行 check.sh 确认测试失败; 随后定位并修复。
- 编写最小 Makefile, 包含 check、build 和 clean; check 调用 check.sh, build 生成 wheel。
- 运行 make check 和 make build, 计算 wheel 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

图 2.14: 第 16 题题目要求

1. 复制项目并初始化 Git 仓库

在 assignment-4 目录中把第 13 题项目复制为 q16, 进入目录后初始化 Git 仓库, 配置本仓库使用的用户名和邮箱, 再把原始文件提交为第一次提交。

```
cp -r q13 q16
cd q16
git init
git config user.name "xiaoyang1088"
git config user.email "3413851324@qq.com"
git add .
```

```
git commit -m "commit1"
```



```
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4$ cp -r q13 q16
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4$ cd q16
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q16$ ls
ai_log.md  check.sh  dist  pyproject.toml  src  tests
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q16$ git init
hint: Using 'master' as the name for the initial branch. This default branch
      name
hint: is subject to change. To configure the initial branch name to use in a
      ll
hint: of your new repositories, which will suppress this warning, call:
hint:   git config --global init.defaultBranch <name>
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command
      :
hint:   git branch -m <name>
Initialized empty Git repository in /home/xy_ai/系统工具开发基础/assignment-4/q16/.git/
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q16$ git config user.name "xiaoyang1088"
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q16$ git config user.email "3413851324@qq.com"
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q16$ git add .
(base) xy_ai@Lenovo: ~/系统工具开发基础/assignment-4/q16$ git commit -m "commit1"
[master (root-commit) 25e5238] commit1
16 files changed, 79 insertions(+)
create mode 100644 ai_log.md
create mode 100755 check.sh
create mode 100644 dist/greetlab_25020007140-0.1.0-py3-none-any.whl
create mode 100644 dist/greetlab_25020007140-0.1.0.tar.gz
create mode 100644 pyproject.toml
create mode 100644 src/greetlab/__init__.py
create mode 100644 src/greetlab/__pycache__/__init__.cpython-312.pyc
create mode 100644 src/greetlab/__pycache__/cli.cpython-312.pyc
create mode 100644 src/greetlab/cli.py
create mode 100644 src/greetlab_25020007140.egg-info/PKG-INFO
create mode 100644 src/greetlab_25020007140.egg-info/SOURCES.txt
create mode 100644 src/greetlab_25020007140.egg-info/dependency_links.txt
create mode 100644 src/greetlab_25020007140.egg-info/entry_points.txt
create mode 100644 src/greetlab_25020007140.egg-info/top_level.txt
create mode 100644 tests/__pycache__/test_cli.cpython-312-pytest-9.1.1.pyc
create mode 100644 tests/test_cli.py
```

图 2.15: 复制 q13、初始化 Git 并提交原始文件

2. 检查初始状态

运行 `git status`，终端显示工作区干净。随后运行 `./check.sh`，Ruff 格式检查、代码检查和两个 Pytest 测试全部通过。

```
xy_ai@Lenovo: ~/系统工具开发 × xy_ai@Lenovo: ~/系统工具开发 × + ▾
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git status
On branch master
nothing to commit, working tree clean
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/xy_ai/系统工具开发基础/assignment-4/q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ cat src/greetlab/cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()

    if not a.name.strip():
        raise SystemExit(2)

    print(f"Hello, {a.name}!")
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ nano src/greetlab/cli.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ ./check.sh
4 files already formatted
F541 [*] f-string without any placeholders
--> src/greetlab/cli.py:12:11
10 |         raise SystemExit(2)
11 |
12 |         print(f"Hello, name!")
   |                ^^^^^^^^^^^^^
help: Remove extraneous `f` prefix
11 |
-     print(f"Hello, name!")
```

图 2.16: 确认初始工作区干净并运行质量检查

3. 制造问题并确认检查失败

把 `src/greetlab/cli.py` 中的问候语临时改成始终输出 `Hello, name!`，再运行 `./check.sh`。Ruff 报告 F541 错误，指出 f-string 中没有占位符，检查以失败结束。

```
print(f"Hello, name!")
```

```
xy_ai@Lenovo: ~/系统工具开发
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ ./check.sh
4 files already formatted
E541 [*] f-string without any placeholders
--> src/greetlab/cli.py:12:11
10 |         raise SystemExit(2)
11 |
12 |         print(f"Hello, name!")
    |         ~~~~~^~~~~~
help: Remove extraneous `f` prefix
11 |
-   print(f"Hello, name!")
12 +   print("Hello, name!")

Found 1 error.
[*] 1 fixable with the `--fix` option.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ nano src/greetlab/c
li.py
```

图 2.17: 修改问候语后质量检查失败

4. 定位并修复问题

根据错误位置重新编辑 cli.py，恢复使用参数中的姓名：

```
print(f"Hello, {a.name}!")
```

再次运行 ./check.sh，Ruff 检查和两个 Pytest 测试全部通过。

```
xy_ai@Lenovo: ~/系统工具开发
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/xy_ai/系统工具开发基础/assignment-4/q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ nano Makefile
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ make check
./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/xy_ai/系统工具开发基础/assignment-4/q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.2
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
```

图 2.18: 修复问候语后重新通过质量检查

5. 编写最小 Makefile

创建 Makefile, 包含 check、build 和 clean 三个目标, 并把它们声明为 PHONY:

```
.PHONY: check build clean

check:
    ./check.sh

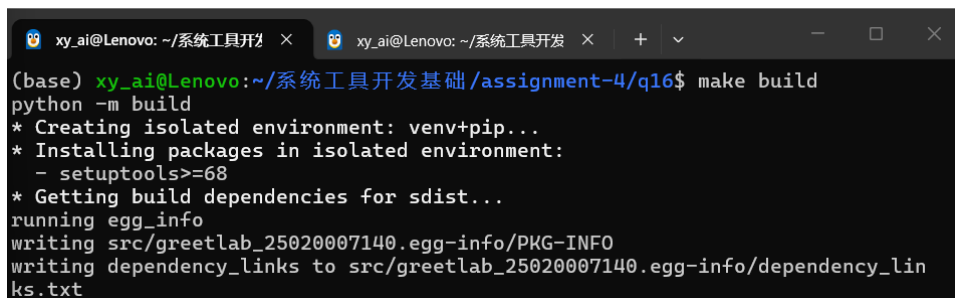
build:
    python -m build

clean:
    rm -rf dist/ build/ *.egg-info
```

运行 `make check` 后, 脚本中的三项检查全部通过。

6. 构建 Wheel

运行 `make build`, Make 调用 `python -m build`, 构建程序创建隔离环境并生成源码包和 Wheel 文件。



```
xy_ai@Lenovo: ~/系统工具开发 x xy_ai@Lenovo: ~/系统工具开发 x + v
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
running egg_info
writing src/greetlab_25020007140.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25020007140.egg-info/dependency_links.txt
```

图 2.19: 使用 `make build` 构建 Python 软件包

7. 计算校验值并提交最终改动

使用 `ls -l dist/*.whl` 检查 Wheel, 再运行 `sha256sum dist/*.whl` 计算 SHA-256。截图中的 Wheel 校验值为:

```
79f4162b3ecc44a4480ace19f15fba5497d9645112cf1a6d132c34e1012b7ce6
```

随后把 Makefile 加入暂存区并提交为第二次提交 `commit2`。该提交只包含 Makefile, 没有混入构建产物或其他临时修改。

```
xy_ai@Lenovo: ~/系统工具开发
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ ls -l dist/*.whl
-rw-r--r-- 1 xy_ai xy_ai 1806 Sep 14 17:13 dist/greetlab_25020007140-0.1.0-py3-none-any.whl
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ sha256sum dist/*.whl
79f4162b3ecc44a4480ace19f15fba5497d9645112cf1a6d132c34e1012b7ce6 dist/greetlab_25020007140-0.1.0-py3-none-any.whl
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   dist/greetlab_25020007140-0.1.0-py3-none-any.whl
    modified:   dist/greetlab_25020007140-0.1.0.tar.gz
    modified:   src/greetlab/cli.py
    modified:   src/greetlab_25020007140.egg-info/SOURCES.txt
    modified:   tests/__pycache__/test_cli.cpython-312-pytest-9.1.1.pyc

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Makefile

no changes added to commit (use "git add" and/or "git commit -a")
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git add Makefile
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git commit -m "commit2"
[master 3036c4f] commit2
 1 file changed, 10 insertions(+)
 create mode 100644 Makefile
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   dist/greetlab_25020007140-0.1.0-py3-none-any.whl
    modified:   dist/greetlab_25020007140-0.1.0.tar.gz
    modified:   src/greetlab/cli.py
    modified:   src/greetlab_25020007140.egg-info/SOURCES.txt
    modified:   tests/__pycache__/test_cli.cpython-312-pytest-9.1.1.pyc

no changes added to commit (use "git add" and/or "git commit -a")
```

图 2.20: 计算 Wheel 的 SHA-256 并提交 Makefile

8. 清理构建造成的工作区改动

构建和测试会改变仓库中原来已跟踪的 Wheel、源码包、SOURCES.txt 和缓存文件。使用 `git restore` 分别恢复这些文件,并恢复临时修改过的 `cli.py`。最后再次运行 `git status`, 终端显示 `nothing to commit, working tree clean`。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git restore src/greetlab/cli.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git restore dist/greetlab_25020007140-0.1.0-py3-none-any.whl
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git restore dist/greetlab_25020007140-0.1.0.tar.gz
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git restore src/greetlab_25020007140.egg-info/SOURCES.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git restore tests/__pycache__/test_cli.cpython-312-pytest-9.1.1.pyc
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$ git status
On branch master
nothing to commit, working tree clean
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/q16$
```

图 2.21: 恢复构建产物后确认 Git 工作区干净

Chapter 3

课后习题

3.1 第 1 题使用 Shell 创建并查看文件

题目要求

在当前目录下完成以下操作：

1. 创建目录 `shell_practice` 并进入该目录。
2. 创建文件 `hello.txt`。
3. 向文件中依次写入 `apple`、`banana` 和 `orange`，每项占一行。
4. 使用 `cat` 查看文件内容。
5. 使用 `pwd` 显示当前所在目录。
6. 使用 `ls -l` 查看文件的详细信息。

要求记录使用的命令和最终输出。

操作过程

1. 创建第一题目录

进入 `assignment-4`，创建 `homework/01` 目录并进入该目录：

```
cd /home/xy_ai/系统工具开发基础/assignment-4
mkdir -p homework/01
cd homework/01
```

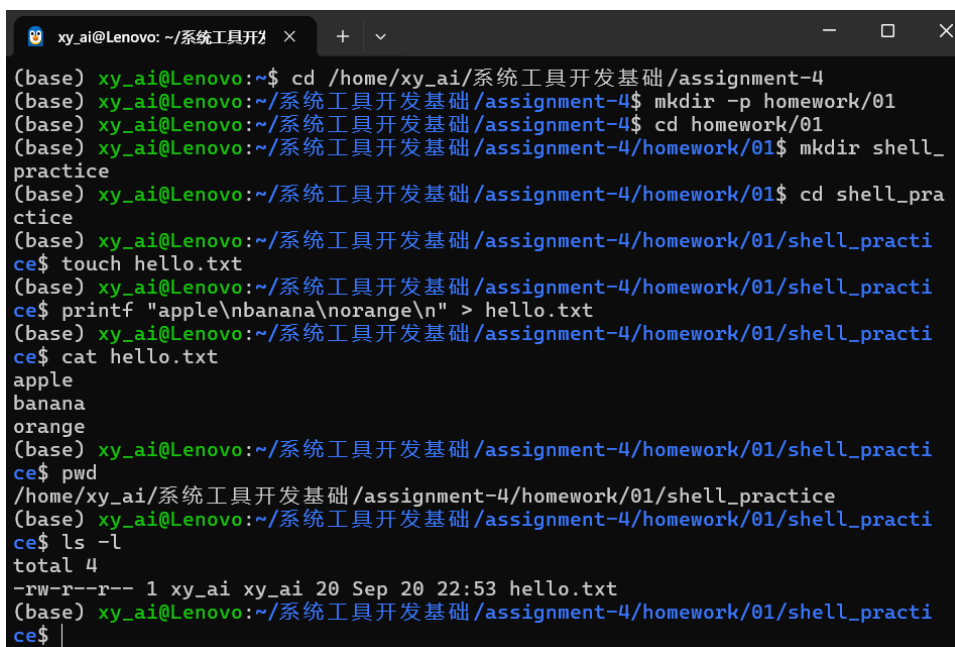
2. 创建文件并写入内容

创建 `shell_practice` 目录并进入，再建立 `hello.txt`。使用 `printf` 向文件中写入三行内容：

```
mkdir shell_practice
cd shell_practice
touch hello.txt
printf "apple\nbanana\norange\n" > hello.txt
```

3. 查看文件内容和目录信息

使用 `cat hello.txt` 查看文件，终端依次输出 `apple`、`banana` 和 `orange`。随后运行 `pwd`，确认当前目录为 `homework/01/shell_practice`。最后使用 `ls -l` 查看文件信息，结果中可以看到 `hello.txt`，文件大小为 20 字节。

A terminal window titled 'xy_ai@Lenovo: ~/系统工具开发' showing a series of commands and their outputs. The user navigates to a directory, creates a subdirectory 'shell_practice', enters it, creates a file 'hello.txt', and writes 'apple', 'banana', and 'orange' to it using printf. Then, they use cat to view the file, pwd to check the current directory, and ls -l to list the file details, which shows 'hello.txt' as 20 bytes.

```
xy_ai@Lenovo: ~/系统工具开发
(base) xy_ai@Lenovo:~$ cd /home/xy_ai/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/01
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/01
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ mkdir shell_practice
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ cd shell_practice
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01/shell_practice$ touch hello.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01/shell_practice$ printf "apple\nbanana\norange\n" > hello.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01/shell_practice$ cat hello.txt
apple
banana
orange
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01/shell_practice$ pwd
/home/xy_ai/系统工具开发基础/assignment-4/homework/01/shell_practice
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01/shell_practice$ ls -l
total 4
-rw-r--r-- 1 xy_ai xy_ai 20 Sep 20 22:53 hello.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01/shell_practice$
```

图 3.1: 创建 `hello.txt` 并查看文件内容和目录信息

3.2 第 2 题使用管道统计文本内容

题目要求

创建文件 `names.txt`，内容如下：

```
Tom
Alice
Tom
Bob
```

```
Alice
Tom
```

完成以下任务：

1. 使用 `sort` 对姓名进行排序。
2. 使用 `uniq -c` 统计每个姓名出现的次数。
3. 使用管道把排序和统计操作连接起来。
4. 将统计结果保存到 `result.txt`。
5. 使用 `cat result.txt` 检查最终结果。

操作过程

1. 创建文本文件

进入 `assignment-4/homework/02` 目录，使用 `printf` 创建 `names.txt`，再用 `cat` 检查文件内容：

```
printf "Tom\nAlice\nTom\nBob\nAlice\nTom\n" > names.txt
cat names.txt
```

文件中共有六行姓名，其中 Tom 出现三次，Alice 出现两次，Bob 出现一次。

2. 对姓名进行排序和统计

先运行 `sort names.txt` 查看按字母顺序排列的结果。由于 `uniq -c` 只能统计相邻的重复行，因此将两个命令通过管道连接：

```
sort names.txt | uniq -c
```

终端显示 Alice 出现 2 次、Bob 出现 1 次、Tom 出现 3 次。

3. 保存并检查统计结果

使用输出重定向把管道的统计结果写入 `result.txt`，再查看文件内容：

```
sort names.txt | uniq -c > result.txt
cat result.txt
```

result.txt 中的内容与终端直接显示的统计结果一致。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/02
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ printf "Tom\nAlice\nTom\nBob\nAlice\nTom\n" > names.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ cat names.txt
Tom
Alice
Tom
Bob
Alice
Tom
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ sort names.txt
Alice
Alice
Bob
Tom
Tom
Tom
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ sort names.txt | uniq -c
      2 Alice
      1 Bob
      3 Tom
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ sort names.txt | uniq -c > result.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ cat result.txt
      2 Alice
      1 Bob
      3 Tom
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ |
```

图 3.2: 使用管道统计姓名出现次数并保存结果

3.3 第 3 题查看环境变量和命令返回码

题目要求

在 Bash 中完成以下操作：

1. 使用 `echo $SHELL` 查看当前 Shell。
2. 使用 `echo $PATH` 查看命令搜索路径。
3. 使用 `which python3` 查看 Python 解释器的位置。
4. 使用 `ls` 查看一个存在的目录，再通过 `echo $?` 查看返回码。
5. 使用 `ls` 查看一个不存在的目录，再次通过 `echo $?` 查看返回码。
6. 简单说明返回码为 0 和非 0 时分别表示什么。

操作过程

1. 查看 Shell 和命令搜索路径

进入 `assignment-4/homework/03` 目录，分别查看当前 Shell 和 PATH 环境变量：

```
echo $SHELL
echo $PATH
```

终端显示当前 Shell 为 `/bin/bash`。PATH 中包含 Miniconda、Linux 系统目录和从 Windows 环境继承的多个命令搜索路径。

2. 查看 Python 解释器的位置

使用下面的命令检查运行 `python3` 时实际调用的解释器：

```
which python3
```

输出为 `/home/xy_ai/miniconda3/bin/python3`，说明当前使用的是 Miniconda base 环境中的 Python 解释器。

3. 比较成功和失败命令的返回码

先查看当前目录并立即读取上一条命令的返回码：

```
ls .
echo $?
```

终端输出返回码 0，表示 `ls .` 执行成功。随后查看一个不存在的目录：

```
ls /not-exist-25020007140
echo $?
```

`ls` 提示目录不存在，返回码为 2。这说明返回码 0 表示命令执行成功，非 0 返回码表示命令执行失败，具体数值可以反映不同的错误情况。

```

(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ cd /home/xy_ai/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/03
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/03
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$
echo $SHELL
/bin/bash
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ echo $PATH
/home/xy_ai/miniconda3/bin:/home/xy_ai/miniconda3/condabin:/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA app/NvDLISR:/mnt/c/Program Files/dotnet:/mnt/c/Program Files (x86)/Windows Kits/10/Windows Performance Toolkit:/mnt/c/Program Files/mingw64/bin:/mnt/c/Program Files/TortoiseSVN/bin:/mnt/d/Git/Git/cmd:/mnt/d/Git/Nodev:/mnt/d/ProgramData/anaconda3/condabin:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/MATLAB/R2026a/bin:/mnt/c/Program Files/Docker/Docker/resources/bin:/mnt/c/Program Files/HZVPN/cli:/mnt/c/Users/34138/Desktop/学术/Python/Scripts:/mnt/c/Users/34138/Desktop/学术/Python:/mnt/c/Users/34138/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/34138/AppData/Local/JetBrains/Toolbox/scripts:/mnt/c/Program Files/JetBrains/PyCharm 2024.1.7/bin:/mnt/c/Users/34138/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/34138/AppData/Roaming/npm:/mnt/c/Users/34138/AppData/Local/Programs/MiKTeX/miktex/bin/x64:/mnt/c/Users/34138/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/34138/AppData/Local/Microsoft/WinGet/Packages/ajeetsouza.zoxide_Microsoft.Winget.Source_8wekyb3d8bbwe:/mnt/c/Users/34138/mingw16/mingw64/bin:/mnt/c/Users/34138/AppData/Local/Microsoft/WinGet/Packages/BurntSushi.ripgrep.MSVC_Microsoft.Winget.Source_8wekyb3d8bbwe/ripgrep-15.2.0-x86_64-pc-windows-msvc:/snap/bin
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ which python3
/home/xy_ai/miniconda3/bin/python3
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ ls .
.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ echo $?
0
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ ls /not-exist-25020007140
ls: cannot access '/not-exist-25020007140': No such file or directory
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ echo $?
2
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ |

```

图 3.3: 查看环境变量、Python 位置和命令返回码

3.4 第 4 题使用 Vim 编辑文本文件

题目要求

使用 Vim 完成以下操作：

1. 创建并打开 vim_practice.txt。
2. 在文件中输入三行文字。
3. 返回正常模式并删除其中一行。
4. 使用撤销命令恢复被删除的内容。
5. 保存文件并退出 Vim。
6. 使用 cat 检查文件的最终内容。

要求说明进入插入模式、返回正常模式、撤销以及保存退出分别使用了什么按键。

操作过程

1. 创建并打开文本文件

进入第 4 题目录，使用 Vim 创建并打开 `vim_practice.txt`：

```
vim vim_practice.txt
```

2. 在 Vim 中输入并编辑文本

打开文件后按 `i` 进入插入模式，依次输入 `First line`、`Second line` 和 `Third line`。输入完成后按 `Esc` 返回正常模式。

在正常模式中输入 `gg` 回到第一行，再按 `j` 移动到第二行。输入 `dd` 删除第二行，然后按 `u` 撤销删除，第二行恢复为 `Second line`。最后输入 `:wq` 并按回车，保存文件并退出 Vim。

3. 检查保存后的文件内容

回到终端后使用下面的命令显示行号并检查文件：

```
cat -n vim_practice.txt
```

终端显示文件仍包含完整的三行内容，说明删除操作已经成功撤销，修改也已经保存。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ cd /home/xy_ai/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/04
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/04
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/04$ vim vim_practice.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/04$ cat -n vim_practice.txt
 1 First line
 2 Second line
 3 Third line
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/04$ |
```

图 3.4: 使用 Vim 编辑文件并检查最终内容

3.5 第 5 题根据报错修复 Python 程序

题目要求

创建 `divide.py`，初始内容如下：

```
def divide(a, b):  
    return a / b  
  
print(divide(10, 0))
```

完成以下任务：

1. 运行程序并记录出现的错误类型。
2. 根据异常信息找到发生错误的代码。
3. 修改程序，使除数为 0 时输出提示信息，而不是直接崩溃。
4. 再次运行程序，确认程序能够正常结束。
5. 简单说明错误产生的原因和修改方法。

操作过程

1. 创建程序并复现错误

在 assignment-4 下创建 homework/05 目录，进入目录后使用 Nano 创建题目给出的 divide.py，然后运行程序：

```
mkdir -p homework/05  
cd homework/05  
nano divide.py  
python3 divide.py
```

程序执行 divide(10, 0) 时，在 return a / b 处触发 ZeroDivisionError: division by zero。原因是 Python 不允许数字除以 0。

2. 增加除数为零的判断

再次打开 divide.py，在执行除法前检查参数 b。修改后的函数如下：

```
def divide(a, b):  
    if b == 0:  
        return "Error: division by zero!"  
    return a / b
```

当除数为 0 时，函数直接返回错误提示；其他情况下仍执行原来的除法运算。

3. 重新运行并检查结果

保存文件后再次运行：

```
python3 divide.py
```

终端输出 `Error: division by zero!`，程序没有再出现异常回溯，能够正常结束。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/05
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/05
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ nano divide.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ python3 divide.py
Traceback (most recent call last):
  File "/home/xy_ai/系统工具开发基础/assignment-4/homework/05/divide.py", line 5, in <module>
    print(divide(10, 0))
    ~~~~~
  File "/home/xy_ai/系统工具开发基础/assignment-4/homework/05/divide.py", line 2, in divide
    return a / b
    ~~~~
ZeroDivisionError: division by zero
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ nano divide.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ python3 divide.py
Error: division by zero!
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$
```

图 3.5: 复现除零错误并检查修复后的运行结果

3.6 第 6 题使用 time 查看 Python 程序运行时间

题目要求

创建 `timer.py`，内容如下：

```
import time

print("Start")
time.sleep(1)
print("End")
```

完成以下操作：

1. 直接运行程序，确认终端依次输出 `Start` 和 `End`。
2. 使用 `time python3 timer.py` 再次运行程序。
3. 记录命令输出中的 `real`、`user` 和 `sys` 时间。
4. 简单说明为什么 `real` 时间接近 1 秒，而 `user` 和 `sys` 时间通常很短。

操作过程

1. 创建计时程序

进入 assignment-4，创建 homework/06 目录并进入，然后使用 Nano 创建题目给出的 timer.py：

```
cd /home/xy_ai/系统工具开发基础/assignment-4
mkdir -p homework/06
cd homework/06
nano timer.py
```

程序先输出 Start，使用 time.sleep(1) 暂停约 1 秒，最后输出 End。

2. 直接运行程序

使用下面的命令运行程序：

```
python3 timer.py
```

终端依次输出 Start 和 End，两次输出之间可以观察到约 1 秒的等待，说明程序能够正常运行。

3. 使用 time 测量运行时间

在 Python 命令前加上 time，记录整个程序的运行时间：

```
time python3 timer.py
```

本次运行的 real 时间为 1.022 秒，user 和 sys 时间均为 0.011 秒。real 表示程序从开始到结束经过的实际时间，其中包含 sleep(1) 的等待时间，因此接近 1 秒；user 和 sys 分别表示用户态和内核态占用 CPU 的时间。程序在暂停期间几乎不使用 CPU，所以这两项都很短。

```
(base) xy_ai@Lenovo:~$ cd /home/xy_ai/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/06
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/06
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ nano timer.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ python3 timer.py
Start
End
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ time python3 timer.py
Start
End

real    0m1.022s
user    0m0.011s
sys     0m0.011s
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ |
```

图 3.6: 运行计时程序并使用 time 查看运行时间

3.7 第 7 题创建并安装简单的 Python 包

题目要求

创建一个名为 `hello_package` 的简单 Python 包，要求如下：

1. 创建项目配置文件 `pyproject.toml`。
2. 创建 `hello_package` 目录和 `hello_package/__init__.py`。
3. 在 `__init__.py` 中编写 `say_hello()` 函数，使其返回字符串 `Hello!`。
4. 创建并激活独立的 Python 虚拟环境。
5. 使用 `pip install .` 安装当前项目。
6. 离开项目源码目录后导入 `hello_package`，并调用 `say_hello()`。
7. 确认终端输出 `Hello!`。

操作过程

1. 创建项目配置和包目录

在 `assignment-4` 下创建 `homework/07` 目录，再建立 `hello_package` 包目录和 `__init__.py`：

```
mkdir -p homework/07
cd homework/07
mkdir hello_package
touch hello_package/__init__.py
```

随后创建 `pyproject.toml`，将项目名称设置为 `hello-package`，版本设置为 `0.1.0`，并使用 `setuptools` 作为构建后端。在 `hello_package/__init__.py` 中编写 `say_hello()` 函数，使其返回字符串 `Hello!`。

2. 创建并激活虚拟环境

使用 Python 自带的 `venv` 模块创建虚拟环境并激活：

```
python3 -m venv .venv
source .venv/bin/activate
which python
```

`which python` 返回当前项目中 `.venv/bin/python` 的完整路径，说明后续安装命令会在独立虚拟环境中执行。

3. 安装并检查软件包

在项目根目录安装当前软件包，再查看其元数据：

```
pip install .
pip show hello-package
```

pip 根据 `pyproject.toml` 构建 Wheel，并成功安装 `hello-package-0.1.0`。pip show 显示包名、版本、说明和虚拟环境中的安装位置，均与项目配置一致。

4. 在源码目录外验证导入结果

返回上一级 `homework` 目录，在仍然激活的虚拟环境中运行：

```
cd ..
python -c "import hello_package; print(hello_package.say_hello())"
```

终端输出 `Hello!`。程序离开项目源码目录后仍能导入并调用该包，说明安装已经生效。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ cd /home/xy_ai/系统工具开发基础/assignment-4
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/07
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/07
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ mkdir hello_package
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ touch hello_package/__init__.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ nano pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ nano hello_package/__init__.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ python3 -m venv .venv
python3 -m venv .venv
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ python3 -m venv .venv
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ source .venv/bin/activate
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ which python
/home/xy_ai/系统工具开发基础/assignment-4/homework/07/.venv/bin/python
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ pip install .
Processing /home/xy_ai/系统工具开发基础/assignment-4/homework/07
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: hello-package
  Building wheel for hello-package (pyproject.toml) ... done
  Created wheel for hello-package: filename=hello_package-0.1.0-py3-none-any.whl size=1255 sha256=bf7272749c4d79ccf749af2c0abb774e7a1f31088e2c8
    Stored in directory: /tmp/pip-ephem-wheel-cache-1cds8dg0/wheels/d9/d5/03/670bd5ac8266de505c0fe527213b200e022ae914685e305758
Successfully built hello-package
Installing collected packages: hello-package
Successfully installed hello-package-0.1.0
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ pip show hello-package
Name: hello-package
Version: 0.1.0
Summary: A simple hello package
Home-page:
Author:
Author-email:
License:
Location: /home/xy_ai/系统工具开发基础/assignment-4/homework/07/.venv/lib/python3.12/site-packages
Requires:
Required-by:
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ cd ..
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ python -c "import hello_package; print(hello_package.say_hello())"
Hello!
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ |
```

图 3.7: 创建、安装并在源码目录外验证 Python 包

3.8 第 8 题使用编程智能体生成 README

题目要求

创建 `calculator.py`，内容如下：

```
def add(a, b):
    return a + b
```

```
print(add(2, 3))
```

向编程智能体提交以下任务：

阅读 `calculator.py`，为这个程序创建 `README.md`，说明程序功能、运行命令和预期输出。不要修改 `calculator.py`。

完成后进行以下检查：

1. 检查智能体是否只创建了 `README.md`。
2. 检查 `README` 是否包含程序功能、运行方法和输出结果。
3. 手动运行 `calculator.py`，核对 `README` 中的说明。
4. 记录提交给智能体的完整提示词。

操作过程

1. 创建并运行示例程序

进入第 8 题目录，并编写题目给出的 `calculator.py`。运行程序后，终端输出 5；使用 `ls` 检查时，目录中只有 `calculator.py`。

```
mkdir -p homework/08
cd homework/08
nano calculator.py
python3 calculator.py
ls
```

```
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd /home/xy_ai/系统工具开发基础/assignment-4
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ mkdir -p homework/08
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/08
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ nano calculator.py
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ python3 calculator.py
5
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ ls
calculator.py
```

图 3.8: 创建并运行 `calculator.py`

2. 向编程智能体提交任务

启动 OpenAI Codex，并说明目标和文件修改范围。提交的提示内容如下：

请读取当前目录中的 `calculator.py`，为这个程序创建 `README.md`。
`README` 需要说明程序功能、`add` 函数、运行命令和预期输出。不要修改 `calculator.py`，也不要创建其他文件。完成后请重新读取 `README`，检查内容是否完整。

Codex 读取当前目录和程序内容后，确认 `add(a, b)` 返回 `a + b`，程序运行时会输出 5，然后开始生成说明文件。

```
>_ OpenAI Codex (v0.155.1)

model:      gpt-6-astra medium  /model to change
directory: ~

Tip: Try the Desktop app. Run 'codex app' or visit
https://chatgpt.com/codex?app-landing-page=true

> 你看看 /home/xy_ai/系统工具开发基础/assignment-4/homework/08，请读取当前目录中的 calculator.py，为这个程序创建 README.md。
README.md 需要说明程序功能、add 函数、运行命令和预期输出。
不要修改 calculator.py，也不要创建其他文件。完成后请重新读取
README.md，检查内容是否完整。

▲Falling back from WebSockets to HTTPS transport. request timed out

• 我会先读取 calculator.py 和目录中的适用说明，再创建 README.md，最后重新读取检查内容是否完整。

✓You approved codex to run wsl.exe -d Ubuntu --exec bash -lc 'cd /home/xy_ai/系统工具开发基础/assignment-4/homew... this time

• Ran wsl.exe -d Ubuntu --exec bash -lc 'cd /home/xy_ai/系统工具开发基础/assignment-4/homework/08 && pwd && ls -la && cat calculator.py; for p
in /AGENTS.md /home/AGENTS.md /home/xy_ai/AGENTS.md /home/xy_ai/系统工
... +5 lines
- /home/xy_ai/系统工具开发基础/assignment-4/homework/08
total 12
... +6 lines (ctrl + t to view transcript)

print(add(2, 3))

• 程序中的 add(a, b) 返回 a + b，运行脚本会打印 5。我会将功能、函数说明、运行命令和预期输出写入 README，并回读核对。

✓You approved codex to run wsl.exe -d Ubuntu --exec python3 -c "from
pathlib import Path; import hashlib... this time
```

图 3.9: 向 Codex 提交 README 生成任务

3. 由智能体创建并检查 README

Codex 创建 `README.md` 后重新读取文件，并检查源程序的校验结果与目录文件。智能体报告 `calculator.py` 没有发生变化，目录中只有 `calculator.py` 和新建的 `README.md`。

`README` 包含程序功能、`add` 函数的参数与返回值、运行命令和预期输出 5，符合题目要求。

```

• 程序中的 add(a, b) 返回 a + b, 运行脚本会打印 5。我会将功能、函数说明、运行命令和预期输出写入 README, 并回读核对。

✓ You approved codex to run wsl.exe -d Ubuntu --exec python3 -c "from pathlib import Path; import hashlib... this time

• Ran wsl.exe -d Ubuntu --exec python3 -c "from pathlib import Path; import hashlib; p=Path('/home/xy_ai/系统工具开发基础/assignment-4/homework/08'); c=p/'calculator.py'; ... +33 lines
  # calculator.py 使用说明

  ... +35 lines (ctrl + t to view transcript)
  CHECK: calculator.py unchanged
  FILES: README.md, calculator.py

• 已在指定目录创建 README.md, 并重新读取检查, 内容完整包含:

- 程序功能
- add 函数的参数、返回值和示例
- 运行命令
- 预期输出 5

已核对 calculator.py 未被修改, 未创建其他文件。

```

图 3.10: Codex 创建 README 并核对文件改动

4. 人工检查生成结果

退出智能体后, 使用 `cat README.md` 查看说明文件, 再查看 `calculator.py` 并重新运行程序。README 的说明完整, 源程序仍保持原样, 再次运行后终端仍输出 `5`。

```

cat README.md
cat calculator.py
python3 calculator.py

```

```
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ cat README.md
# calculator.py 使用说明

## 程序功能

这是一个简单的 Python 加法程序。程序定义 `add` 函数，并调用 `add(2, 3)`，将计算结果打印到终端。

## add 函数

```python
def add(a, b):
 return a + b
```

- 参数: `a` 和 `b` 是参与加法运算的两个值。
- 返回值: `a + b` 的计算结果。对于数字参数，返回两数之和。
- 示例: `add(2, 3)` 返回 `5`。

## 运行命令

需要安装 Python 3。在终端执行：

```bash
cd /home/xy_ai/系统工具开发基础/assignment-4/homework/08
python3 calculator.py
```

如果终端已在该目录，只需执行 `python3 calculator.py`。

## 预期输出

```text
5
```

程序通过 `print(add(2, 3))` 输出上述结果。
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ cat calculator.py
def add(a, b):
    return a + b

print(add(2, 3))
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ python3 calculator.py
5
```

图 3.11: 人工检查 README、源程序和运行结果

3.9 第 9 题编写简单的 Bug 报告

题目要求

创建 `list_bug.py`，内容如下：

```
numbers = [1, 2, 3]
print(numbers[5])
```

运行程序并根据结果创建 `BUG_REPORT.md`。报告至少包括以下内容：

1. 操作系统和运行环境。
2. Python 版本。
3. 运行程序所使用的命令。
4. 期望结果。
5. 实际结果。
6. 程序显示的错误类型。

7. 可以复现该问题的具体步骤。

操作过程

1. 创建程序并复现错误

进入 assignment-4/homework/09 目录，创建题目给出的 list_bug.py。先查看 Python 版本，再运行程序：

```
python3 --version
python3 list_bug.py
```

本次使用的版本为 Python 3.12.4。程序中的列表只有三个元素，却尝试访问下标为 5 的元素，因此在第 2 行触发 `IndexError: list index out of range`，程序异常结束。

2. 编写 Bug 报告

创建 BUG_REPORT.md，按照题目要求记录运行环境、执行命令、期望结果、实际结果、错误类型和复现步骤。报告中的主要内容如下：

```
# Bug Report

## 环境
- 系统: WSL2 Ubuntu
- Python: Python 3.12.4

## 运行命令
`python3 list_bug.py`

## 期望结果
程序正常输出列表中的数字，或者在下标无效时给出提示并正常结束。

## 实际结果
程序异常结束，终端显示 `IndexError: list index out of range`。

## 错误类型
`IndexError`
```

复现步骤写明进入目录、确认程序文件、运行命令以及出现错误的位置，其他人按照这些步骤即可重新观察到相同问题。

3. 检查报告内容

使用 `cat BUG_REPORT.md` 查看最终文件。终端中显示的环境信息、错误类型和复现步骤与实际运行结果一致，报告包含了定位和复现问题所需的主要信息。

```
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4$ cd homework/09
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ nano list_bug.py
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ python3 --version
Python 3.12.4
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ python3 list_bug.py
Traceback (most recent call last):
  File "/home/xy_ai/系统工具开发基础/assignment-4/homework/09/list_bug.py", line 2, in <module>
    print(numbers[5])
          ~~~~~^~~
IndexError: list index out of range
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ nano BUG_REPORT.md
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ cat BUG_REPORT.md
# Bug Report

## 环境
- 系统: WSL2 Ubuntu
- Python: Python 3.12.4

## 运行命令
`python3 list_bug.py`

## 期望结果
程序正常输出列表中的数字, 或者在下标无效时给出提示并正常结束。

## 实际结果
程序异常结束, 终端显示 `IndexError: list index out of range`。

## 错误类型
`IndexError`

## 复现步骤
1. 进入 `assignment-4/homework/09` 目录。
2. 确认目录中存在 `list_bug.py`。
3. 运行 `python3 list_bug.py`。
4. 程序在访问 `numbers[5]` 时出现 `IndexError`。
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ |
```

图 3.12: 复现列表越界错误并编写 Bug 报告

3.10 第 10 题为加法函数编写单元测试

题目要求

创建 calculator.py, 内容如下:

```
def add(a, b):
    return a + b
```

再创建 test_calculator.py, 使用 Pytest 完成以下任务:

1. 测试 add(2, 3) 的返回值是否等于 5。
2. 测试 add(-1, 1) 的返回值是否等于 0。
3. 使用 pytest -q 运行测试, 确认两项测试全部通过。
4. 故意把其中一个期望值改错, 观察测试失败信息。
5. 恢复正确的期望值并重新运行测试, 确认测试再次通过。

操作过程

1. 创建函数和测试文件

进入第 10 题目录并建立 `calculator.py`。文件中的 `add(a, b)` 函数直接返回两个参数之和。

随后创建 `test_calculator.py`，分别测试两个正数相加和一正一负相加：

```
from calculator import add

def test_add_positive():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-1, 1) == 0
```

2. 运行正确测试

使用下面的命令运行测试：

```
python3 -m pytest -q
```

Pytest 自动发现两个测试函数，终端显示 `2 passed in 0.01s`，说明加法函数同时通过两个测试用例。

3. 故意修改期望值并观察失败

把第二个测试中的正确期望值 0 临时改为 1：

```
def test_add_negative():
    assert add(-1, 1) == 1
```

再次运行测试后，终端显示 `1 failed, 1 passed`。失败信息指出函数实际返回 0，但测试期望得到 1，因此触发 `AssertionError`。

4. 恢复测试并再次验证

将第二个断言的期望值恢复为 0，再次执行 `python3 -m pytest -q`。终端重新显示 `2 passed in 0.01s`，说明测试文件已恢复正确，两个用例均能通过。

```

(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir 10
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 10
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ nano calculator.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ python3 -m pytest -q
..
2 passed in 0.01s [100%]
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ nano test_calculator.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ python3 -m pytest -q
.F [100%]
===== FAILURES =====
----- test_add_negative -----
def test_add_negative():
>     assert add(-1, 1) == 1
E     assert 0 == 1
E     + where 0 = add(-1, 1)

test_calculator.py:9: AssertionError
===== short test summary info =====
FAILED test_calculator.py::test_add_negative - assert 0 == 1
1 failed, 1 passed in 0.04s
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ nano test_calculator.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ python3 -m pytest -q
..
2 passed in 0.01s [100%]
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ |

```

图 3.13: 运行正确测试、故意制造失败并恢复测试

Chapter 4

解题感悟

4.1 课上练习

1. 第 13 题中，我学会了把代码格式、静态检查和自动化测试放进同一个脚本。运行脚本后可以一次检查多个项目要求，并通过返回值判断检查是否全部成功。
2. 第 14 题中，我学会了在 Makefile 中写明文件之间的依赖关系。源文件没有变化时 Make 不会重复生成产物，依赖文件更新后才会重新执行对应命令。
3. 第 15 题中，我学会了使用 curl 获取本地 API 数据，并用 jq 完成筛选、排序和格式转换。把这些命令写进脚本后，可以重复生成内容一致的 Markdown 报告。
4. 第 16 题中，我练习了从检查仓库状态开始，经过制造问题、定位修复、重新测试、构建 Wheel 和提交改动的完整过程。最后还要清理构建产生的修改，保持 Git 工作区干净。

4.2 课后习题

1. 第 2 题让我学会了用管道连接命令。先排序再统计，比手动数姓名更快，结果也能直接保存到文件。
2. 第 5 题先保留除零报错，再增加判断并重新运行。这个过程让我明白，修复程序不能只改代码，还要用相同输入确认错误已经消失，最后的结果才更可靠。

3. 第 8 题让我发现，给智能体说明任务时，既要写清要生成什么，也要限制不能修改哪些文件。README 完成后，我又查看源程序并重新运行，确认文件没有变化，结果仍然正确。这样比只看回复更可靠。
4. 第 10 题中，正确测试、故意失败和恢复测试形成了清楚的对照。我能直接看到断言怎样发现错误，也明白测试通过后还要检查修改是否合理。

Chapter 5

上传 GitHub

5.1 编写并更新.gitignore

提交第四次实验前，我继续在课程仓库根目录更新 `.gitignore`。本次实验会产生 Python 虚拟环境、缓存目录、构建目录、`egg-info` 元数据和 LaTeX 编译中间文件。这些文件都可以重新生成，没有必要全部上传。第四次实验使用的主要忽略规则如下：

```
# LaTeX 编译中间产物
*.aux
*.fdb_latexmk
*.fls
*.log
*.out
*.synctex.gz
*.xdv
*.toc

# Python 缓存和测试工具缓存
__pycache__/.
*.py[cod]
.pytest_cache/
.ruff_cache/
.venv/

# 第四次实验的 Python 构建中间产物
/assignment-4/**/*.egg-info/
/assignment-4/**/*.build/
```

```
# 第四次实验报告的临时文件和备份
/assignment-4/实验报告/tmp/
/assignment-4/实验报告/*.bak
```

其中，`.venv`、Python 缓存和测试缓存都可以重新创建；`build` 和 `egg-info` 属于构建过程中产生的文件。LaTeX 的辅助文件、临时检查图片和备份文件也没有必要提交。`dist` 目录中的发布包保留为实验结果。这样处理后，仓库中主要保留源码、测试、作业截图和正式实验报告。

5.2 GitHub 仓库链接

本课程作业保存在下面的公开仓库中，默认分支为 `main`：

https://github.com/xiaoyang1088/system_development_tools

仓库用户名为 `xiaoyang1088`，仓库名为 `system_development_tools`。第四次实验位于 `assignment-4/` 目录，其中包括课堂练习 `q13-q16`、课后练习 `homework/`、实验截图和实验报告。

5.3 commit 提交记录

第四次实验分五次提交。

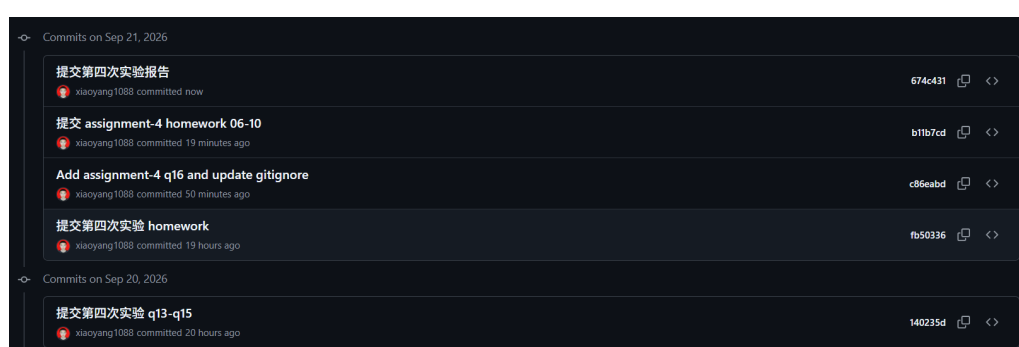


图 5.1: 第四次实验作业的 GitHub commit 提交记录